

«Командная строка Windows», работа № 11.

Лабораторный практикум дисциплины «Операционные системы».

Для того чтобы сдать лабораторную работу, все примеры, приводимые в теоретическом материале, должны быть представлены преподавателю в виде созданных студентом bat – файлов.

Перенаправление ввода/вывода и конвейеризация (композиция) команд

Рассмотрим более подробно поддерживаемые в Windows UNIX-подобные концепции переназначения устройств стандартного ввода/вывода и конвейерного выполнения команд.

С помощью переназначения устройств ввода/вывода одна программа может направить свой вывод на вход другой или перехватить вывод другой программы, используя его в качестве своих входных данных. Таким образом, имеется возможность передавать информацию от процесса к процессу при минимальных программных издержках. Практически это означает, что для программ, которые используют стандартные входные и выходные устройства, операционная система позволяет:

- выводить сообщения программ не на экран (стандартный выходной поток), а в файл или на принтер (перенаправление вывода);
- читать входные данные не с клавиатуры (стандартный входной поток), а из заранее подготовленного файла (перенаправление ввода);
- передавать сообщения, выводимые одной программой, в качестве входных данных для другой программы (конвейеризация или композиция команд).

Из командной строки эти возможности реализуются следующим образом. Для того, чтобы перенаправить текстовые сообщения, выводимые какой-либо командой, в текстовый файл, нужно использовать конструкцию

команда > имя_файла

Если при этом заданный для вывода файл уже существовал, то он перезаписывается (старое содержимое теряется), если не существовал — создается. Можно также не создавать файл заново, а дописывать информацию, выводимую командой, в конец существующего файла. Для этого команда перенаправления вывода должна быть задана так:

команда >> имя_файла

С помощью символа < можно прочесть входные данные для заданной команды не с клавиатуры, а из определенного (заранее подготовленного) файла:

команда < имя_файла

Приведем несколько примеров перенаправления ввода/вывода.

1. Вывод встроенной справки для команды COPY в файл copy.txt
COPY /? > copy.txt

2. Добавление текста справки для команды XCOPY в файл copy.txt:

```
XCOPY /? >> copy.txt
```

3. Ввод новой даты из файла date.txt (DATE — это команда для просмотра и изменения системной даты):

```
DATE < date.txt
```

4. Копирование с клавиатуры в файл

```
COPY CON myfile.txt
```

При выполнении этой команды данные с клавиатуры (стандартное устройство CON - консоль) будут записываться в файл myfile.txt. Нажатие клавиши F6 или комбинации CTRL-Z завершит вывод.

Если при выполнении определенной команды возникает ошибка, то сообщение об этом по умолчанию выводится на экран. В случае необходимости сообщения об ошибках (стандартный поток ошибок) можно перенаправить в текстовый файл с помощью конструкции

команда 2> имя_файла

В этом случае стандартный вывод будет производиться на экран. Также имеется возможность информационные сообщения и сообщения об ошибках выводить в один и тот же файл. Делается это следующим образом:

команда > имя_файла 2>&1

Например, в приведенной ниже команде стандартный выходной поток и стандартный поток ошибок перенаправляются в файл copy.txt:

```
XCOPY A:\1.txt C: > copy.txt 2>&1
```

Наконец, с помощью конструкции

команда1 | команда2

можно использовать сообщения, выводимые первой командой, в качестве входных данных для второй команды (конвейер команд).

Используя механизмы перенаправления ввода/вывода и конвейеризации, можно из командной строки посылать информацию на различные устройства и автоматизировать ответы на запросы, выдаваемые командами или программами, использующими стандартный ввод. Для решения таких задач подходит команда

ECHO [сообщение]

которая выводит сообщение на экран. Рассмотрим примеры использования этой команды.

1. Посылка символа прогона на принтер:

```
ECHO ^L > PRN
```

2. Удаление всех файлов в текущем каталоге без предупреждения (автоматический положительный ответ на запрос об удалении):

```
ECHO y | DEL *.*
```

3. Соединение по телефону из командной строки (модем связан с портом COM2):

ЕCHO ATDT 1(123)555-1234 > COM2

Команды **MORE** и **SORT**

Одной из наиболее часто используемых команд, для работы с которой применяется перенаправление ввода/вывода и конвейеризация, является **MORE**. Эта команда считывает стандартный ввод из конвейера или перенаправленного файла и выводит информацию частями, размер каждой из которых не больше размера экрана. Используется **MORE** обычно для просмотра длинных файлов. Возможны три варианта синтаксиса этой команды:

```
MORE [диск:] [путь]имя_файла
MORE < [диск:] [путь]имя_файла
имя_команды | MORE
```

Параметр **[диск:] [путь]имя_файла** определяет расположение и имя файла с просматриваемыми на экране данными. Параметр **имя_команды** задает команду, вывод которой отображается на экране (например, **DIR** или команда **TYPE**, используемая для вывода содержимого текстового файла на экран). Приведем два примера.

Для постраничного просмотра помощи команды **DIR** используется команда:

```
DIR /? | MORE
```

Для постраничного просмотра текстового файла `news.txt` возможны следующие варианты команд:

```
MORE news.txt
MORE < news.txt
TYPE news.txt | MORE
```

Другой распространенной командой, использующей перенаправление ввода/вывода и конвейеризацию, является **SORT**. Эта команда работает как фильтр: она считывает символы в заданном столбце, упорядочивает их в возрастающем или убывающем порядке и выводит отсортированную информацию в файл, на экран или другое устройство. Возможны два варианта синтаксиса этой команды:

```
SORT [/R] [/+n] [[диск1:] [путь1] файл1] [> [диск2:] [путь2] файл2]
```

или

```
[команда] | SORT [/R] [/+n] [> [диск2:] [путь2] файл2]
```

В первом случае параметр **[диск1:] [путь1] файл1** определяет имя файла, который нужно отсортировать. Во втором случае будут отсортированы выходные данные указанной команды. Если параметры **файл1** или команда не заданы, то **SORT** будет считывать данные с устройства стандартного ввода.

Параметр **[диск2:] [путь2] файл2** задает файл, в который будет направляться сортированный вывод; если этот параметр не задан, то вывод будет направлен на устройство стандартного вывода.

По умолчанию сортировка выполняется в порядке возрастания. Ключ **/R** позволяет изменить порядок сортировки на обратный (от Z к A и затем от 9 до 0). Например, для постраничного просмотра отсортированного в обратном порядке файла price.txt, нужно задать следующую команду:

```
SORT /R < price.txt |MORE
```

Ключ **/+n** задает сортировку в файле по символам n-го столбца. Например, **/+10** означает, что сортировка должна осуществляться, начиная с 10-й позиции в каждой строке. По умолчанию файл сортируется по первому столбцу.

Исполняемые (командные) файлы Windows

Введение

В операционной системе MS-DOS и во всех операционных системах семейства Microsoft Windows простейшим средством автоматизации обработки файлов (и каталогов) служат так называемые командные файлы. Командные файлы — не единственное средство автоматизации работы пользователя даже из числа встроенных. В операционных системах семейства Microsoft Windows, начиная с версии 98, имеется еще Windows Script(ing) Host. Кроме того в этих целях может быть использован интерпретируемый язык Perl, Python или Ruby. Перечисленные средства, достаточно мощны, но требуют умения программировать, т.е. составлять и, что немаловажно, отлаживать программы, хотя бы на неплохом любительском уровне. Использование же командных файлов доступно всякому без особенной подготовки, вполне достаточно внимательности и здравого смысла.

Командный процессор

В состав многих операционных систем, в том числе, разработанных корпорацией Microsoft, входит командный процессор. Так называется программа, которая инициирует выполнение всевозможных действий в ответ на команды, вводимые пользователем с клавиатуры. В основном эти действия заключаются в запуске нужных программ с теми или иными параметрами. Но некоторые команды выполняются непосредственно командным процессором. В основном это те команды, которые служат для управления контекстом и последовательностью выполнения команд. Любая программа, которую технически возможно запустить из командной строки, рассматривается командным процессором как команда. Он не делает различий между «родными» командами, изначально встроенными в операционную систему, и программами, которые были установлены поверх нее.

Командная строка и команды

Окно командного процессора в первоизданном виде выглядит мрачно, и работать с ним большинству людей не очень удобно. Намного проще пользоваться файловыми менеджерами в стиле Norton Commander или FAR. В них предусмотрены как средства для быстрой навигации по файловой системе, так и строка для ввода команд.

Перенаправление ввода-вывода

Результат выполнения команды **help** не умещается на один экран (та же проблема возникает с текстом описания команды **for**). Это неудобство можно избежать благодаря перенаправлению выдачи результата в файл.

Задание №1

Для того что бы перенаправить результат выполнения команды в файл используется символ '>':

1. Откройте FAR (через меню Пуск, или введя команду far в «Запуске программы»)
2. Перейдите в вашу рабочую папку (в FARе: нажмите Alt+F1 и далее в свою папку)
3. Нажмите сочетание клавиш Ctrl+O (переход к командой строке, чтобы вернуться в оболочку FAR, нажмите Ctrl+O снова)
4. Наберите текст команды **help > a.txt** в командной строке.
5. Нажмите на клавишу **Enter**.
6. Посмотрите результат: Откройте через файловый менеджер FAR вновь созданный файл a.txt.
7. Наберите текст команды **help >> a.txt** в командной строке.
8. Нажмите на клавишу **Enter**.
9. Посмотрите результат: Откройте через файловый менеджер FAR вновь созданный файл a.txt (выберите соотв. файл и нажмите клавишу F4).
10. Определите в чем разница использования '>' и '>>' (Для этого соответствующие команды можно выполнить несколько раз, при этом каждый раз необходимо оценивать полученный результат)?
11. Ознакомьтесь со списком доступных команд.

Задание №2

Ознакомьтесь с тем как работают команды **move** и **ren**:

1. Изучите справку по этим командам.
2. Создайте в вашем рабочем каталоге каталоги **box** и **table**.
3. В каталоге box создайте файл test1.txt.
4. Переместите файл test1.txt из каталога box в каталог table с помощью команды: **move box\test1.txt table**.
5. Переименуйте файл test1.txt в файл test2.txt с помощью команды: **ren test1.txt test2.txt**.

Текущий каталог. Абсолютные и относительные пути

При работе с файловыми командами исключительную важность приобретает понятие текущего каталога. Дело в том, что при указании файла в качестве параметра команды мы всегда используем один из двух возможных способов указания на них: либо абсолютный путь, либо относительный путь. В полном пути мы указываем все, начиная с диска (или сетевого имени компьютера), например **d:\misha\box\test1.txt**. Какой бы каталог ни оказался текущим в момент ввода команды, полный путь будет

соответствовать одному и тому же файлу. Для относительного пути текущий каталог служит отправной точкой. Простейший случай относительного пути — имя файла. В контексте выполнения команды оно означает файл с таким именем, расположенный в текущем каталоге.

Для записи относительного пути к текущему каталогу существует условная запись. (точка). Для записи относительного пути к каталогу, в котором содержится текущий каталог, существует условная запись `..` (две точки). Команда `copy *.* .\table` копирует все файлы из текущего каталога в каталог **table**, расположенный рядом с ним.

Командные файлы

До сих пор, приводя примеры, мы считали, что всякий раз вводим команды вручную. При обработке большого количества файлов или при систематическом выполнении одних и тех же команд это становится обременительно. Поэтому в командном процессоре предусмотрена возможность выполнения командных файлов. **Командный файл — это текстовый файл, в котором набраны команды** (или хотя бы одна команда). Пример командного файла:

```
help copy > copy.help
help move > move.help
md msdos-help
move *.help msdos-help
```

Задание №3

1. Объясните, что происходит в ходе выполнения указанных команд.

Командным файлам принято давать расширение **bat**. По нему файлы этого типа опознаются командным процессором. Этот файл можно назвать, например, **make-help.bat**.

Для того чтобы запустить командный файл:

1. Введите его имя в качестве команды. После этого командный файл будет выполнен.

В командном файле каждая команда занимает одну строку. Точнее, существует способ расположить одну команду на нескольких подряд идущих строках, для этого непосредственно перед каждым переводом строки следует поставить символ «крышка» `^`. (Необходимо, чтобы каждая «крышка» была последним символом в своей строке; после нее не должно быть пробелов и табуляций). Пример:

```
if exist Z:\test.txt ^
copy test.txt ^
Z:\test\
```

При выполнении командного файла командный процессор просматривает его сверху вниз от первой строки к последней и выполняет команды в том порядке, в котором их обнаруживает. Выполняет он в целом их так, как если бы каждая из них вводилась вручную.

При необходимости последовательность выполнения команд можно изменить с помощью управляющих.

Не путайте текущий каталог с каталогом, в котором находится запускаемый командный файл. Предположим, текущим каталогом является **work**, в нем находится каталог **tools**, а в каталоге **tools** хранятся командные файлы. Вы запускаете один из них командой **tools\collect-images.bat**. Так вот, «с точки зрения» этого командного файла, текущим каталогом все равно окажется **work**, а не **tools**.

Комментирование командного файла и его выдачи.

Команды **echo** и **rem**

Командный файл, по существу, представляет собой программу, написанную на языке командного процессора операционной системы. Текст программы полагается снабжать комментариями, чтобы, вернувшись к нему некоторое время спустя, не вспоминать мучительно, для чего эта программа нужна, и как она устроена.

В системе команд MS-DOS для оформления комментариев предусмотрена команда **rem**. Это фиктивная команда, которая не предполагает выполнения каких бы то ни было действий, но позволяет написать в строке после своего имени произвольный текст. Причем командный процессор не воспринимает его как синтаксическую ошибку.

Пример:

```
rem *****  
rem Формирование файлов справки по командам copy и move  
rem *****  
  
rem Формируем файлы справки  
help copy > copy.help  
help move > move.help  
  
rem Создаем каталог для хранения файлов справки  
md msdos-help  
  
rem Перемещаем файлы справки в подготовленный каталог  
move *.help msdos-help
```

Обратите внимание на пустые строки, которыми командный файл разбит на «абзацы». Этот простой прием позволяет сделать командный файл более читабельным.

При выполнении приведенного выше командного файла все команды будут выводиться на экран по мере их выполнения, что не всегда удобно. Выдачу команд можно отключить с помощью команды **@echo off**. Символ «собака» перед командой **echo** означает, что и сама эта команда должна выполняться в «молчаливом» режиме. С таким же успехом мы могли бы не пользоваться командной **echo off**, а поместить «собаку» перед каждой командой.

Во многих случаях требуется, чтобы командный файл выводил на экран (или в файл) те или иные сообщения. В одних случаях это могут быть сообщения об ошибках, в других информационные сообщения, объясняющие пользователю командного файла, что происходит в данный момент. Для вывода сообщений применяется та же самая команда

echo. В качестве параметра ей передают текст выводимого сообщения. Пример усовершенствованного командного файла:

```
@echo off

rem *****
rem Формирование файлов справки по командам copy и move
rem *****

@echo Формируем файлы справки. Одну секундочку...

rem Формируем файлы справки
help copy > copy.help
help move > move.help

rem Создаем каталог для хранения файлов справки
md msdos-help

rem Перемещаем файлы справки в подготовленный каталог
move *.help msdos-help

echo Готово!
```

Передача командному файлу параметров

Предположим, мы хотим создать командный файл, который сначала формирует справку с описанием заданной пользователем команды, а потом загружает его для просмотра в блокнот. Фокус в том, чтобы при очередном запуске командного файла каким-то образом сообщить ему, какая именно команда нас интересует на этот раз.

Для решения этой задачи предусмотрен механизм обработки параметров. Работает он довольно просто. Если при запуске командного файла пользователь указал несколько параметров, то в тексте командного файла первый из них мы обозначаем записью %1, второй записью %2, третий записью %3 и т.д. Этими обозначениями мы пользуемся в тексте командного файла примерно так же, как в естественной речи местоимениями.

Текст командного файла, решающего поставленную задачу, приведен ниже. Обратите внимание на команду **help**. В качестве ее параметра ей передается первый параметр командного файла.

```
@echo off

rem Формируем файл с описанием команды,
rem имя которой передано параметром
help %1 > help.tmp

rem Загружаем файл описания в редактор Блокнот
notepad help.tmp
```

Предположим, что мы присвоили этому командному файлу имя **show-help.bat**. Для того чтобы загрузить в блокнот описание команды, например, **dir**, мы должны ввести команду следующим образом: **show-help.bat dir**

Следующий командный файл создает каталог с именем, указанным в первом параметре, и записывает в него файл с текстом описания команды, указанной во втором параметре:

```
rem Пример командного файла с двумя параметрами

rem Создаем каталог с именем, заданным первым параметром
md C:%1

rem Создаем в нем файл с описанием команды,
```



```
rem заданной вторым параметром
help %2 > %1\%2.help
```

Что произойдет, если пользователь при запуске этого командного файла укажет не два, а четыре параметра? Ничего страшного, они ничему не помешают, просто не будут использованы. А что будет, если пользователь укажет только первый параметр? Вторым параметром окажется пустым. Эффект получится такой: командный файл будет выполнен, но так, как будто на месте записи %2 ничего нет. Команда **help** сформирует список всех команд и поместит его в файл с пустым именем и **расширением .help**. Если же пользователь запустит этот файл, не указав ни одного параметра, то при попытке командного процессора выполнить команду **md** (предназначена для создания каталога), будет выдано сообщение о синтаксической ошибке, поскольку у команды **md** обязательно должен быть параметр.

Таким образом, использование параметров создает большие возможности, но может существенно усложнить дело. Для того чтобы командный файл всегда работал корректно, необходимо проверять корректность указания пользователем параметров и каким-то образом реагировать на неполные или неверные входные данные.

Переменные. Команда set

Переменной называется поименованное значение. Для объявления переменной и одновременно для присвоения ей значения применяется команда **set**. Пример записи этой команды показан на следующем листинге.

```
rem Компилятор хелп-файлов в формате CHM
set help_compiler=C:\Program Files\HTML Help Workshop\hhc.exe
```

Для извлечения значения переменной ее имя помещают между двумя знаками процента, как показано ниже.

```
rem Компилятор хелп-файлов в формате CHM
set help_compiler=C:\Program Files\HTML Help Workshop\hhc.exe
```

```
rem Проект хелп-файла модуля "Склад"
set store_hpj="Z:\store.hpj"
```

```
rem Проект хелп-файла модуля "Продажи"
set sales_hpj="Z:\sales.hpj"
```

```
rem Компилируем хелп-файлы
%help_compiler% %store_hpj%
%help_compiler% %sales_hpj%
```

Данный пример показывает, в чем польза переменных.

Во-первых, они позволяют вместо длинного фрагмента (например, пути к компилятору хелп-файлов) использовать внутри командного файла синонимичный ему короткий – это удобно.

Во-вторых, они позволяют избежать повторения в тексте командного файла фрагментов, которые в дальнейшем могут меняться.

Параметры командного файла тоже являются переменными, но от обычных переменных отличаются тем, что их значения задаются при запуске командного файла. В дальнейшем, говоря о переменных, в особенности о работе с их значениями, мы будем

иметь в виду и параметры командного файла тоже, во всяком случае, в отсутствие явных оговорок на это счет.

При написании командных файлов часто применяют следующий прием: несколько значений переменных указывают рядом (или перемежая их какими-либо символами или строками), так, чтобы получить некоторое новое осмысленное значение.

Пример:

```
rem Путь к компилятору хелп-файлов
set help_compiler=" C:\Program Files\HTML Help Workshop\hhc.exe "

rem Путь к каталогу, в котором находятся проекты хелп-файлов
set project_path="Z:\<ваша_папка>\help"
rem Вызываем компилятор для обработки конкретного проекта,
rem имя которого передаем в первом параметре
%help_compiler% %project_path%\%1.hpj
```

Проверка условий и выбор вариантов. Команды if и goto

Команда **if** позволяет выделять в командном файле группы команд, которые выполняются или не выполняются в зависимости от определенных условий. Для чего это нужно?

Проверка условия — почти необходимая мера при создании командных файлов, использующих параметры. Перед тем, как начинать работу, командный файл, вообще говоря, должен удостовериться в том, что ему передан корректный набор параметров. В противном случае велик риск, что он выполнится неверно или безрезультатно, а пользователю останется только гадать, в чем же проблема. Более того, если командный файл удаляет, перемещает или перезаписывает какие-либо данные, то при некорректных параметрах он может даже нанести ущерб.

На следующем примере показан уже знакомый вам командный файл компиляции хелп-файла. В начало командного файла добавлена проверка первого параметра на непустоту. Обратите внимание на такую особенность синтаксиса: для операции сравнения используется двоянный знак равенства. Если первый параметр оказывается непустым, срабатывает команда **goto**, которая «перебрасывает» командный процессор к указанной метке. В данном случае имя этой метки **compile**. Обратите внимание, что там, где метка находится, ее имя предваряется двоеточием, а в команде **goto** нет. При пустом первом параметре командный процессор переходит к следующей строке, которая выдает сообщение об ошибке. А потом к следующей, которая перебрасывает его в самый конец файла к метке с именем **finish**.

```
@echo off
```

```
rem Проверяем, задан ли параметр
if not "%1"==" " goto compile
```

```
rem Если параметр пуст, выдаем сообщение об ошибке
echo Не указано имя проекта хелп-файла
rem и переходим в конец командного файла
rem к метке finish
goto finish
```

```
rem Это метка с именем compile
:compile
```

```
rem Ниже расположены команды компиляции
```

```

rem Путь к компилятору хелп-файлов
set help_compiler="c:\Program Files\HTML Help Workshop\hhc.exe"

rem Путь к каталогу, в котором находятся проекты хелп-файлов
set project_path="Z:\<ваша_папка>\help"

rem Вызываем компилятор для обработки конкретного проекта,
rem имя которого передаем в первом параметре
%help_compiler% %project_path%\%1.hpj

rem Это метка с именем finish
:finish

```

Предложенный способ проверки параметра не самый удачный.

Во-первых, если пользователь по ошибке укажет в качестве параметра имя несуществующего файла, командный файл этим удовлетворится и предпримет попытку компиляции. Более правильный способ — проверить, существует ли такой файл в действительности. Для этого в языке команд MS-DOS предусмотрено специальное слово **exist**. Поэтому лучше было бы написать: **if exist %1.hpj goto compile**.

Во-вторых, активное использование команды **goto** (т.н. безусловного перехода) и меток сильно запутывают код. Технически они ничем не плохи, но отлаживать и сопровождать командный файл, написанный в таком стиле, довольно неудобно. Поэтому программисты издавна считают безусловный переход приемом нежелательным. Ниже показан более правильный, с точки зрения стиля программирования, структурированный вариант, в котором используется конструкция **if...else**. Работает она так: если условие истинно, выполняются команды в скобках после **if**, а если ложно, то в скобках после **else**.

```

@echo off

rem Проверяем, задан ли параметр
if not exist %1.hpj (

    rem Если параметр пуст, выдаем сообщение об ошибке
    echo Такого проекта хелп-файла не существует.
) else (

    rem Ниже расположены команды компиляции

    rem Путь к компилятору хелп-файлов
    set help_compiler=" C:\Program Files\HTML Help Workshop\hhc.exe"

    rem Путь к каталогу, в котором находятся проекты хелп-файлов
    set project_path="Z:\<ваша_папка>\help"

    rem Вызываем компилятор для обработки конкретного проекта,
    rem имя которого передаем в первом параметре
    %help_compiler% %project_path%\%1.hpj

)

```

Приведем еще один пример работы с проверками. Следующий командный файл создает каталог с именем help-files (предположим, для загрузки в него скомпилированных хелп-файлов). При этом, если каталог с таким именем уже существует (и в нем, вероятно, находятся старые хелп-файлы, которые не хотелось бы терять: вдруг новые окажутся хуже?), командный файл присваивает ему расширение **bak**. Но если каталог **help-files.bak** уже существовал, то командный файл его удаляет (будем считать, что одной резервной копии нам хватит).

```

if exist help-files.bak rd help-files.bak
if exist help-files ren help-files help-files.bak
md help-files

```

Массовая обработка файлов. Команда **for**

Команда **for** позволяет организовать выполнение повторяющихся однотипных действий. Можно использовать ее для того, чтобы вывести на экран числа от одного до десяти, как показано на следующем листинге.

```
for /l %%i in (1,1,10) do echo %%i
```

Переменная **i** называется счетчиком цикла. В силу своеобразия синтаксиса команды **for**, имя счетчика цикла должно состоять из одной буквы. Причем, если пишем командный файл, то перед именем счетчика цикла надо поставить сдвоенный знак процента, если же мы просто набираем команду в командной строке, то одиночный.

Логика работы этой команды такова. После слова **in** указан диапазон изменения счетчика цикла. В данном варианте команды это тройка чисел: начальное значение счетчика, шаг счета, предельное значение счетчика. При выполнении команды командный процессор сначала присвоит переменной **i** значение 1, а потом на каждом шаге цикла будет увеличивать его на 1, пока оно не превысит 10. Очевидно, таких шагов получится десять. Если бы в качестве шага счета мы указали число 2, то цикл выполнялся бы пять раз. На каждом шаге цикла выполняется тело цикла, написанное после слова **do**. В приведенном примере это команда **echo**, которая выводит на экран текущее значение счетчика цикла.

Наверно можно придумать ситуацию, когда что-то подобное на самом деле требуется, но обычно команда **for** используется для перебора и обработки файлов. Надо сказать, что в достаточно простых случаях массовая обработка файлов выполняется с помощью подстановочных символов. Если, мы хотим всем файлам в текущем каталоге заменить расширение **.htm** на **.html**, мы вводим команду **ren *.htm *.html**. Но если то же самое надо сделать не в одном каталоге, а в дереве каталогов, то без команды **for** не обойтись. Приведенный ниже командный файл выполняет эту операцию для всех **htm-файлов** в подкаталоге **website** текущего каталога. Точнее, во всем дереве каталогов, которое находится внутри **website**.

```
for /r website %%i in (*.htm) do ren %%i %%~ni.html
```

Ключ **/r** указывает на необходимость обхода каталога **website** и всех его внутренностей. Если его не указать (но тогда и каталог указывать не разрешается), то обработаны будут только файлы в текущем каталоге. Диапазоном значений счетчика цикла в данном варианте команды является множество всех файлов с расширением **.htm**, находящихся внутри каталога (точнее, дерева) **website**. Странная на первый взгляд запись **~ni** означает, что из значения переменной **i** требуется выделить только имя файла. В языке команд MS-DOS предусмотрено несколько таких модификаторов, например, запись **~xi** обозначает расширение файла. Все модификаторы описаны в справке по команде **for**.

Тело цикла может состоять из нескольких команд, заключенных в скобки.

```
@echo off
for /r website %%i in (*.htm) do (
    rem Выводим имя файла
    echo %%i
    rem Переименовываем файл
    ren %%i %%~ni.html
)
```

Передача управления другому командному файлу. Команда call

Существует возможность вызвать из одного командного файла другой командный файл. Для этого служит команда **call**. Замечательно, переменные, заданные в вызывающем командном файле «видны» вызванному. И наоборот, после того, как вызванный файл закончит работу и вернет управление вызвавшему, последний будет «видеть» переменные, оставленные ему вызванным «в наследство». Это позволяет разработчику командных файлов действовать, например, следующим образом. Если несколько командных файлов должны пользоваться одними и теми же значениями, допустим, путями к каким-то файлам, их можно вынести в отдельный командный файл, который будет играть роль конфигурационного файла. Каждый рабочий командный файл будет начинаться вызовом конфигурационного. Выигрыш в том, что при изменении путей вносить изменения придется только в один конфигурационный файл, а не во множество рабочих.

«Конфигурационный» командный файл **config.bat**.

```
rem Путь к компилятору хелп-файлов
set help_compiler="C:\Program Files\HTML Help Workshop\hhc.exe"
rem Путь к каталогу, в котором находятся проекты хелп-файлов
set project_path="Z:\<ваша_папка>\help"
```

«Рабочий» командный файл.

```
@echo off
rem Настраиваем переменные
call config.bat
rem Проверяем, задан ли параметр
if not exist %1.hpj (
    rem Если параметр пуст, выдаем сообщение об ошибке
    echo Такого проекта хелп-файла не существует.
) else (
    rem Ниже расположены команды компиляции
    rem Вызываем компилятор для обработки конкретного проекта,
    rem имя которого передаем в первом параметре
    %help_compiler% %project_path%\%1.hpj
)
```

Вопросы:

1. Дайте определение командного файла? Какое расширение имеет командный файл?
2. В чем отличие использования `>` и `>>`, при организации перенаправления вывода?
3. Как перенести длинную строчку в командном файле?
4. Каким образом передаются параметры командному файлу? Приведите пример командного файла с параметрами.
5. Объясните назначение команды **set**. В чем преимущество использования переменных? Приведите пример командного файла, использующего команду **set**.
6. Чем отличаются параметры командного файла от его переменных?
7. В каких случаях может быть использована команда **for**? Приведите пример командного файла, использующего команду **for**.
8. Какая команда применяется для вызова из одного командного файла другого командного файла? Приведите пример подобного командного файла (-ов).

Используйте дополнительный материал для подготовки ответов на следующие вопросы:

1. Что означает: экранировать любой другой символ с безусловным приоритетом.
2. Объясните назначение операторов: `<`, `>`, `>>`, `&`, `&&` и `||`.

Закрепляем навыки по командным файлам

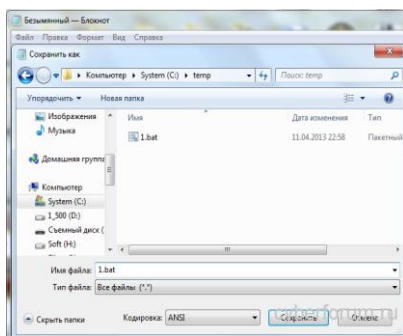
Как создать batch файл?

Вариант 1. Через блокнот или другой редактор

Откройте блокнот и наберите две следующих строки:

```
@echo This batch file  
@pause
```

- Выберите "Файл" -> "Сохранить как...".
- Выберите "Тип файла" -> "Все файлы".
- В поле "Имя файла" введите **1.bat** и сохраните его в папке **Temp** диска **C**, например.



- Теперь откройте эту папку и запустите файл 1.bat. На экран будет выведено две строки:

This batch file

Нажмите любую клавишу для продолжения...

- После нажатия любой клавиши окно закроется, т.к. bat-файл выполнен.

Обратите внимание, что символ собаки перед каждой командой в bat-файле указывает на то, что саму команду выводить на экран не надо, а нужно вывести только результат ее работы. Чтобы поэкспериментировать удалите символ собаки из начала каждой строки, сохраните и запустите полученный bat файл.

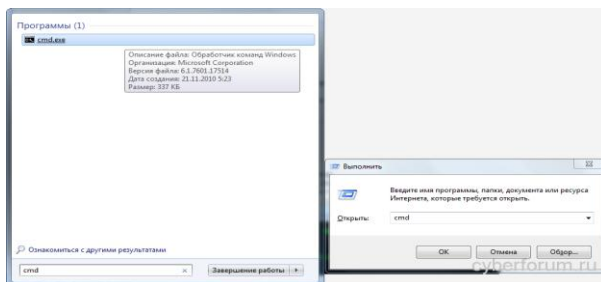
Чтобы не вводить постоянно @, можно задекларировать отключение вывода в самом начале бат-файла, указав **@echo off**:

```
@echo off  
echo This batch file  
pause
```

Вариант 2. Через консоль перенаправлением потока вывода

- Запуск консоли

ПУСК -> Выполнить (либо Win + R), вводим CMD, нажимаем {ENTER}



Откроется маленькое черное окно - консоль (или "интерпретатор командной строки Windows", еще называют "командный процессор").

В нем Вы увидите путь к каталогу, в котором будут выполняться команды в данный момент.

Пишем команды. Ввод каждой строки подтверждаем клавишей {ENTER}.

```
cd /D "c:\temp"
echo @echo This batch file > 1.bat
echo @pause >> 1.bat
1
```

Пояснение:

- 1-й строкой мы переходим в каталог c:\temp
- 2-я строка создает файл 1.bat с текстом @echo This batch file
- 3-я строка дописывает в конец файла текст @pause
- 4-я строка вызывает на исполнение файл 1.bat

Внимание! Для выполнения некоторых команд в Windows Vista, 7, 8 требуются **повышенные права**.

Вариант 3. Через консоль копированием данных с устройства con (клавиатура)

- Запуск консоли (все аналогично способу 2 (см. выше).
- Пишем команду:

```
copy con myproga.cmd
```

- Пишем содержимое нашего бат-файла. Ввод каждой строки подтверждаем клавишей {ENTER}.

```
@echo off
echo This batch file
pause
```

- Нажимаем Ctrl + Z, затем клавишу Enter.
- Запускаем на исполнение созданный бат-файл:

```
myproga
```

Вариант 3.1. Через консоль, используя команду more

- Пишем команду:

more > myproga.cmd

- Пишем содержимое нашего бат-файла. Ввод каждой строки подтверждаем клавишей {ENTER}.

@echo off

echo This batch file

pause

- Нажимаем Ctrl + C (либо Ctrl + Z и клавишу Enter).
- Запускаем на исполнение созданный бат-файл:

myproga

Написав в **блокноте Windows** (Notepad.exe) батник **1.bat** такого содержания

@Echo off

Echo Введите текст. Завершение ввода - комбинация Ctrl+Z {ENTER}

copy con temp.txt

мы получим "абракадабру":

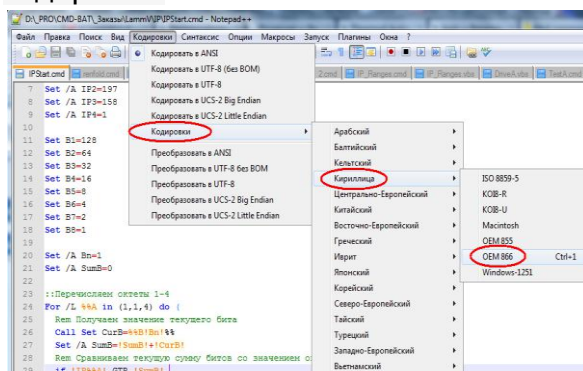
```
C:\Users\enemylo1>C:\Users\enemylo1\Desktop\bat\1.bat
C:\Users\enemylo1>Echo ттхфшЄх ЄхьёЄ. ||ртхЕ°хэшх ттюфр - ъьюсшэрЎш Ctrl+Z <ENTER>
ттхфшЄх ЄхьёЄ. ||ртхЕ°хэшх ттюфр - ъьюсшэрЎш Ctrl+Z <ENTER>
```

Почему так происходит?

Консоль работает в кодировке DOS (codepage 866).

Блокнот Windows создает пакетный файл в кодировке WIN (codepage 1251).

Поэтому, когда Вы запускаете файл, консоль отображает его в своей (DOS) кодировке.



Помните: в редакторе не должно быть видно "крякозябер", иначе это значит: Вы неправильно скопировали код.

В Windows 7 часто бывает, что код, списанный с какого-либо интернет-форума, "портится" в буфере. Выход: когда копируете код, убедитесь, что включена русская раскладка клавиш.

Задачи:

1. Скрипт (**params.bat First Second "Five words"**) выводит на экран свои параметры.
2. Создать два виртуальных диска. Скрипт проверяет наличие двух файлов. Если первый (на первом диске) существует – проверяем присутствие второго на другом диске. Если первого нет – завершаем работу.
3. Скрипт №1 вызывает на работу скрипт №2.
4. Скрипт создает архив файлов, именуемый из текущей даты **ДД.ММ.ГГГГ.rar**.
5. Скрипт создает архив каталога **"Мои документы"** и размещает его в каталоге **C:\ARCHIV**.
6. Скрипт с помощью утилиты **AT.EXE** отрабатывает расписание типа: в 15:32 запускаем **notepad.exe**, в 15:35 напоминаем, что надо позвонить маме, в 15:40 – как насчет чашки чая, в 15:50 – отметку, вероятно, уже поставили – пора домой...
7. Скрипт с помощью команды **NET.EXE**, либо с помощью утилиты **PSService.exe** (из набора **PsTools**) опрашивает состояние службы, например **DNSCache**. И в зависимости от принятого состояния приостанавливает, либо перезапускает эту службу.
8. Скрипт отображает вводимые пользователем символы клавиатуры и ободряет пользователя (все о.к.) до ввода символа 0, либо (также - финал), если 10 секунд ввода не было.
9. Скрипт использует результат работы утилиты **ping.exe**, которая обращалась к несуществующему адресу, и команды **FIND.EXE**, которая ищет в потоке информации слово, например, **TTL**.
10. Скрипт выводит текущую версию Windows на данной локальной машине.
11. Скрипт создает файл **dirlst.txt**, содержащий список каталогов диска C:\.
12. Скрипт находит на диске все файлы с расширением **.log** и перемещает их в каталог **XLAM**.
13. Скрипт запускает два графических приложения (например, **cmd.exe** и **notepad.exe**) и они после запуска не мешают работе друг друга!
14. Скрипт позволяет скрытно работать в Интернете: окно **cmd.exe** закрывает, например, окно **EXPLORER.EXE**.
15. Скрипт преобразует текстовый файл в **DOS-кодировке** в новый текстовый файл в **Windows-кодировке**.